

K. J. SOMAIYA SCHOOL OF ENGINEERING

Somaiya Vidyavihar University, Mumbai

Department of Artificial Intelligence & Data Science

INTERNAL ASSESSMENT: GENERATIVE ARTIFICIAL INTELLIGENCE

Solver-Verified Text-Conditioned Sokoban Level Generation

*Autoregressive Models Can Count,
One-Shot Decoders Cannot*

A comparison of three generative model families (a decoder-only transformer, a conditional variational autoencoder and a conditional generative adversarial network) on the task of generating playable 10×10 Sokoban puzzles from short text captions, evaluated with an **exhaustive A* solver** used as a decision procedure rather than with a learned or heuristic proxy metric.

Submitted by

Name	Roll Number	Class
Aayush Kumar Tiwari	16014223003	LY AI & DS B
Prateeksha Kini	16014223061	LY AI & DS B

Academic Year 2025–26

All code, checkpoints, generated levels and result artifacts accompany this report.

Every numeric claim below is traceable to a committed JSON artifact.

Abstract

Generative models are almost always judged by proxies. Perplexity does not tell you whether a program runs, and a discriminator score does not tell you whether a game level can be finished. This project asks what changes when the proxy is removed entirely.

We generate 10×10 Sokoban puzzles from short natural-language captions such as "hard, ~40 moves, dense walls, corridors, boxes scattered", and we check every single generated level with an **exhaustive A* solver**. Because the state space of a 10×10 level with four boxes is finite and every pruning rule we use is sound, running the search to exhaustion is a *proof* that no solution exists. The verifier is therefore a decision procedure, not a one-sided test, and we report three outcomes (*solved*, *proven unsolvable*, *timed out*) and never collapse them into two.

Three generative families are compared at roughly comparable scale: a 10.7M-parameter decoder-only transformer written from scratch, a 1.1M-parameter conditional VAE, and a 1.1M-parameter conditional GAN, plus a DistilGPT-2 pretraining ablation. The central finding is architectural. A valid level needs *exactly* one player, four boxes and four goals. A transformer that emits the grid cell by cell can count what it has already placed and reaches 82.6% structural validity with no help; a VAE that decodes all 100 cells at once from a single latent vector cannot, and produces **zero** valid levels in 100,000 draws under argmax decoding. Applying a hard constraint to the logits during sampling closes that gap deterministically at 100% validity.

Three further results complicate the simple story. Solvability does not follow from validity: an empty room with no interior walls is already solvable 23.4% of the time against the transformer’s 49.6%. Repairing the VAE’s counting errors gives 76.6% solvability, *above* the transformer, which locates the autoregressive advantage in counting rather than in spatial understanding. And measuring across three training seeds proved essential: solvability moves by ± 7.0 points across seeds while validation loss moves by ± 0.005 , so validation loss is a poor proxy for the property actually being evaluated.

Keywords: procedural content generation, autoregressive models, variational autoencoders, generative adversarial networks, constrained decoding, verification, Sokoban.

Provenance. This project is a replication and extension of Todd et al. (FDG 2023), *Level Generation Through Large Language Models*. The data-scaling finding, the solvability-versus-conditioning setup and the tokenization finding are theirs; we claim none of them. No novelty is claimed anywhere in this report.

Contents

Abstract	ii
1 Introduction	1
1.1 Background and motivation	1
1.2 Problem statement	1
1.3 Objectives	2
1.4 Scope and non-goals	2
1.5 Organisation of this report	2
2 Literature Survey	3
2.1 Procedural content generation via machine learning	3
2.2 Language models for level generation	3
2.3 Verification versus proxy metrics	3
2.4 Sokoban and search	3
2.5 What this project adds	3
3 Dataset and Caption Construction	5
3.1 The Boxoban corpus	5
3.2 Verifying the format instead of assuming it	5
3.3 A data-quality finding: some published solutions are wrong	5
3.4 Caption construction	5
3.5 Final splits and representations	6
4 Generative Models and Constrained Decoding	7
4.1 Task formulation	7
4.2 Transformer (primary system)	7
4.3 Conditional variational autoencoder	8
4.3.1 Two decoding protocols, both reported	8
4.4 Conditional generative adversarial network	8
4.5 DistilGPT-2 ablation: does pretraining transfer?	8
4.6 The counting problem, stated precisely	9
4.7 Constrained decoding: a hard rule on the logits	10
4.7.1 How often does it actually fire?	10
4.7.2 The honest cost	10
4.8 Repair: projecting an invalid grid onto a valid one	11
5 Evaluation Methodology	12
5.1 The solver as a decision procedure	12
5.1.1 Two sound prunes, and nothing else	12
5.2 Solver validation	12

5.2.1	Reconstructed move length is an upper bound	13
5.3	Metrics	13
5.4	Baselines	14
5.5	Statistical protocol	14
6	Implementation	15
6.1	Pipeline	15
6.2	Stack and hardware	15
6.3	Repository layout	15
6.4	Reproducibility	15
6.5	Testing	16
7	Results and Discussion	17
7.1	Main results	17
7.2	Finding 1: autoregressive models can count; one-shot decoders cannot	18
7.3	Finding 2: solvability does not follow from validity	19
7.4	Finding 3: repair separates counting from spatial understanding	20
7.5	Finding 4: controllability splits along a predictable line	20
7.6	Finding 5: temperature trades solvability against diversity	22
7.7	Finding 6: seed variance is large, and it bounds every other claim	22
7.8	Finding 7: pretraining transfers across a vocabulary boundary	23
7.9	Failure analysis: where every family breaks	23
7.9.1	Extrapolation fails completely	24
7.9.2	The one-shot failure mode is a plausible, empty room	24
7.9.3	Hand-written domain knowledge is not a shortcut	24
7.9.4	GAN mode collapse	24
8	Limitations, Ethics and Conclusion	25
8.1	Limitations	25
8.2	Ethical considerations	25
8.3	Conclusion	25
8.4	Future work	26
	References	27
A	Reproduction and Artifacts	28
A.1	What is released	28
A.2	Full reproduction	28
A.3	Statement on numeric provenance	28

List of Figures

1.1	What a Sokoban level is	1
4.1	Why one-shot decoders cannot count	7
4.2	Training curves for all three families	9
4.3	Constrained decoding at a step where it binds	10
5.1	Node-cap convergence	13
6.1	System pipeline	15
7.1	Headline solvability ranking	17
7.2	Tile counts per family	18
7.3	VAE latent interpolation under argmax	19
7.4	Validity and solvability are different axes	20
7.5	Generated levels, one row per family	21
7.6	Temperature Pareto frontier	22
7.7	Seed variance versus validation loss	23

List of Tables

2.1	Where this project sits relative to the closest prior work.	4
3.1	Verified dataset format properties	5
3.2	The five caption features	6
4.1	The four models side by side	9
5.1	Solver validation	12
5.2	The metrics used throughout Chapter 7.	13
6.1	Module layout.	15
7.1	Main results: validity and solvability	17
7.2	Main results: novelty, diversity and cost	18
7.3	Pairwise two-proportion z -tests on solvable valid.	20
7.4	Controllability per attribute	21
7.5	Temperature sweep on the constrained transformer.	22
7.6	Seed variance	22
7.7	Out-of-distribution length requests	24
A.1	Result artifacts and what each one backs.	28

Introduction

1.1 Background and motivation

A generative model is only as trustworthy as the thing that scores it. In text generation we use perplexity; in image generation, FID; in procedural content generation, hand-designed heuristics or a learned discriminator. All of these share a weakness: they measure whether the output *resembles* the training data, not whether it *works*.

Some domains do better. HumanEval runs unit tests on generated code, Lean checks generated proofs, and text-to-SQL is scored by executing the query. So it is simply not true that no metric can tell you whether a generated artifact functions. But every one of those verifiers is **sound but incomplete**: a passing unit test does not prove a program correct, and a failing test suite cannot prove a program wrong; it can only fail to find a counter-example. The verifier answers “yes” reliably and “no” only provisionally.

Sokoban is unusual because it admits the other kind of verifier. A 10×10 grid with four boxes has a state space that is finite and small enough to search exhaustively. Given pruning rules that are *sound*, meaning they discard only states from which no solution could exist, draining the search frontier to empty is a constructive proof that the level is unsolvable. That makes the verifier a **decision procedure**, and it lets us ask a question most generative-model evaluations cannot ask: not “does this look right?” but “is this actually playable, yes or no?”

One real Boxoban level and its solution | caption: "easy, ~25 moves, open room, single chamber, boxes scattered" | 23 moves

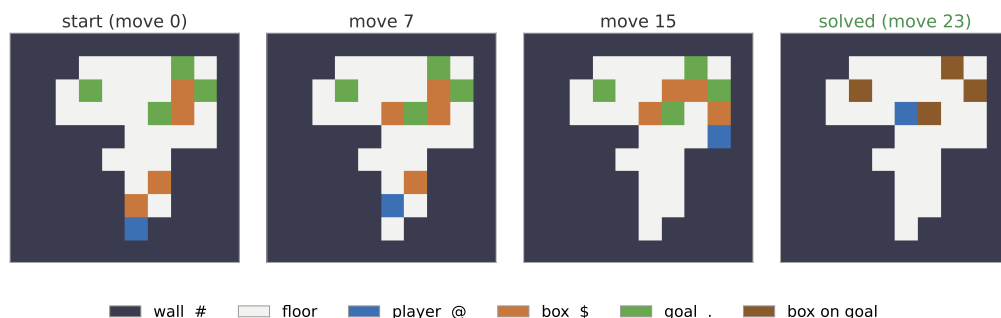


Figure 1.1: Sokoban in one picture. The player (blue) pushes boxes (orange) onto goal squares (green); boxes can only be *pushed*, never pulled, so a box shoved into a corner is lost forever. The level is solved when every box stands on a goal. This is a real held-out Boxoban level, replayed under its published 23-move solution. The caption above it is the kind of text our models are conditioned on, and it is computed from the grid, not hand-written.

1.2 Problem statement

Given a short natural-language caption describing a puzzle’s difficulty, solution length, wall density, corridor structure and box layout, generate a 10×10 Sokoban level that is (i) *structurally valid*, with exactly one player, four boxes, four goals and a complete wall border, (ii) *solvable*, as proven by exhaustive search, (iii) *novel*, not a copy of a training level, and (iv) *faithful to the caption*.

The four requirements pull against each other, and that is the point. Copying a training level maximises solvability and destroys novelty. Generating an empty room gives decent solvability and ignores the caption. A single headline number hides all of this, so the results in this report are always presented as a joint constraint over several axes rather than as one score.

1.3 Objectives

1. Build and *validate* an exhaustive A* Sokoban solver, and establish before anything else is measured that it can be trusted as ground truth.
2. Train three generative model families (autoregressive transformer, conditional VAE, conditional GAN) at roughly comparable parameter counts on the same data with the same conditioning information.
3. Test a specific architectural hypothesis: that autoregressive decoding can satisfy hard counting constraints that one-shot decoding cannot.
4. Implement constrained decoding (a hard logit mask) and grid repair, and report them as separate rows so that *counting* and *spatial understanding* are measured separately rather than merged.
5. Measure controllability per caption attribute, and quantify how far the results move when only the random seed changes.
6. Report negative results such as out-of-distribution failure, mode collapse and inverted rankings as findings rather than as problems to tune away.

1.4 Scope and non-goals

This is an *engineering and evaluation* project, not an architecture proposal. We contribute no new model design. The models are deliberately standard so that the comparison is about the decoding structure, not about clever tricks. Concretely:

- One game, one grid size: 10×10 , four boxes, four goals. Nothing here shows the finding generalises to other tile games or grid sizes.
- All training was done on a single laptop RTX 3050 with 6 GB of VRAM. Model sizes are chosen to fit that budget honestly, not to be competitive with large models.
- Hyperparameters were chosen once and never swept. They are reported as *admitted defaults* throughout rather than justified after the fact.
- No novelty is claimed. Similar pipelines exist in the literature and are cited in Chapter 2.

1.5 Organisation of this report

Chapter 2 surveys related work and states precisely what is borrowed and what is added. Chapter 3 covers the dataset and a set of data-quality findings that turned out to matter. Chapter 4 describes the three generative families and the constrained decoding scheme in which the central hypothesis is tested. Chapter 5 describes the solver and the evaluation protocol, and Chapter 6 the implementation. Chapter 7 presents the results and the failure analysis, and Chapter 8 states the limitations and concludes.

Literature Survey

2.1 Procedural content generation via machine learning

Procedural content generation via machine learning (PCGML) has used autoencoders and GANs over tile grids for years. The standard recipe is to encode a level as a 2-D grid of tile categories, train a generative model over that grid, and evaluate with a mixture of visual inspection, tile-distribution statistics and an agent that tries to play the level. Our models are drawn from exactly this tradition; the contribution is not the architecture but the verifier attached to it.

2.2 Language models for level generation

Todd et al. (FDG 2023) fine-tune GPT-2 variants to generate Sokoban levels from a character encoding and study how solvability scales with the amount of training data and with the choice of tokenization. **This project is a replication and extension of that work.** The data-scaling result, the solvability-versus-conditioning setup and the tokenization finding are theirs, and we claim none of them.

MarioGPT conditions level generation on natural language for *Super Mario Bros.*, checking playability with an A* agent. That agent is a sound but incomplete verifier: it can confirm a level is playable but cannot prove that one is not. The distinction matters for how results may be reported, and it is the specific gap this project addresses.

2.3 Verification versus proxy metrics

The most relevant precedents come from outside PCG: HumanEval executes unit tests against generated programs, interactive theorem provers check generated proofs mechanically, and text-to-SQL benchmarks score by execution accuracy. In every case the verifier is sound but incomplete. A decision procedure, which is what an exhausted search over a finite state space gives us, is strictly stronger, and is rarely available in generative evaluation.

2.4 Sokoban and search

Sokoban is PSPACE-complete in general, and the solver literature is built on domain-specific pruning. Our solver is deliberately conservative and uses only two prunes, both provably sound, because an *unsound* prune produces false proofs of unsolvability, the worst failure mode available here, since it corrupts the headline number invisibly and in a flattering direction. The Boxoban corpus we use was introduced for model-free planning research and generated by reverse play from an already-solved state.

2.5 What this project adds

Relative to the work it replicates, three things are added:

1. **Constrained decoding with feasibility forcing**, which makes structural validity deterministic rather than learned, and lets us separate “can the model count?” from “does the model understand the puzzle?”
2. **Three-way solver reporting** that distinguishes proven unsolvability from timeout, which is only possible because the verifier is a decision procedure.
3. **A controlled multi-family comparison** at comparable parameter counts, including one-shot decoders and four non-learned baselines that bound how the headline number may be interpreted.

Table 2.1: Where this project sits relative to the closest prior work.

Work	Verifier	Relation to this project
Todd et al. 2023	A* agent (sound, incomplete)	Directly replicated and extended. Data-scaling and tokenization findings are theirs.
MarioGPT	A* agent (sound, incomplete)	Same text-conditioning idea, different game; cannot prove unplayability.
PCGML (autoencoders, GANs)	tile statistics, human inspection	Source of our VAE and GAN baselines.
This project	exhaustive A* (decision procedure)	Adds constrained decoding, three-way verdicts, and a controlled multi-family comparison.

Dataset and Caption Construction

3.1 The Boxoban corpus

We use the Boxoban `unfiltered` split: 100,000 training levels, 5,000 from the *official* validation split (we deliberately do not carve a validation set out of train), and the 1,000-level test split held frozen for solver validation and used as the “real levels” ceiling row in every results table. Solution lengths come from a published A* solution set.

3.2 Verifying the format instead of assuming it

Every property the representation depends on was verified rather than assumed. The checks are cheap and each of them would have broken something silently had it failed.

Table 3.1: Format properties verified before any modelling (`results/data_verification.json`).

Property	Finding
Tile alphabet	Exactly {#, @, \$, ., space} over 50 level files. Neither * (box on goal) nor + (player on goal) occurs anywhere , so a box never <i>starts</i> on a goal.
Grid invariants	10,000 of 10,000 levels are 10×10 with a complete wall border and exactly one player, four boxes and four goals. Zero violations.
Line endings	Files are CRLF; the parser normalises them.
Solution units	Steps counts player moves, not pushes : <code>Steps == len(Actions)</code> on 989/989 well-formed rows, with digits 0–3 meaning Up, Right, Down, Left.
Floor components	Every Boxoban level has exactly one connected floor region, measured over 20,000 levels.
Cross-split leakage	Zero collisions between train and either held-out split.

Two of these findings changed design decisions. The absence of * and + is what makes the five-channel one-hot tensor, the six-symbol grid vocabulary and the “exactly four \$ and four .” constraint all well-defined at once. And the floor-component result killed a planned caption feature: “number of connected floor regions” is a *constant* on this corpus, so it carries no information and cannot bin anything. We replaced it with mean floor degree, the average number of non-wall neighbours per floor cell, which ranges from 2.19 to 3.36 and does discriminate.

3.3 A data-quality finding: some published solutions are wrong

The solution set documents rows whose action field is `SEARCH_STATE_FAILED` or `NOT_FOUND`. Beyond those, we found rows that *look* perfectly well-formed, a plain digit string, but that **do not replay to a solved state**: 63 of 1000 (6.3%) in the test split and approximately 0.7% in train. They fail by walking into a wall or making an impossible push part-way through, and treating blocked moves as no-ops rescues none of them; no direction convention or transposition rescues them either.

Every solution used in this project is replay-validated against an independent simulator, and levels whose solution does not replay are dropped: 734 of 100,000 training levels (0.73%), 373 of 5,000 validation levels and 74 of 1,000 test levels (7.4%). A caption built on a wrong solution length is worse than no example at all, because it teaches the model a false association with full confidence.

3.4 Caption construction

Five features are computed from each grid; **nothing is hand-labelled**. The thresholds are the terciles and medians of the training distribution, so the bins are balanced by construction.

Table 3.2: The five caption features and their measured bin boundaries (`results/dataset_build.json`).

Feature	How it is computed	Boundary
Difficulty	solution length, binned by training terciles	$\leq 26 / \leq 35 / > 35$ moves
Solution length	rounded to the nearest five moves	range 4–130
Wall density	interior # count / 64, split at the training median	0.500
Connectivity	mean non-wall neighbours per floor cell, split at the median	2.889
Box clustering	mean pairwise Manhattan distance between boxes, split at the median	3.500

A caption reads, for example:

```
hard, ~40 moves, dense walls, corridors, boxes scattered
```

The same five features are encoded twice: as **caption text** for the transformer and as a **10-dimensional vector** for the VAE and GAN. These are different information channels, so the family comparison is not perfectly controlled on conditioning. We state this rather than claim otherwise; it is listed again in Chapter 8 as a limitation.

3.5 Final splits and representations

After replay validation, 99,266 training rows, 4,627 validation rows and 926 test rows survive. Each row carries the raw grid, the caption text, the 10-dimensional caption vector, and the verified solution length.

Two representations are derived from each grid:

- **Character sequence** (110 characters: ten rows of ten tiles plus ten newlines) for the autoregressive models, wrapped as `<bos> caption <sep> grid <eos>`. Vocabulary is 48 hand-written characters; the longest sequence observed is 176 tokens.
- **Five-channel one-hot tensor** of shape $5 \times 10 \times 10$ for the one-shot decoders, one channel per tile type.

Generative Models and Constrained Decoding

4.1 Task formulation

Let c be a caption and g a level. All four models estimate $p(g | c)$; they differ entirely in *how they factorise it*, and that factorisation is the whole experiment.

$$\text{Autoregressive: } p(g | c) = \prod_{t=1}^{110} p(g_t | g_{<t}, c) \quad (4.1)$$

$$\text{One-shot latent: } p(g | c) = \int p(z | c) \prod_{i=1}^{100} p(g_i | z, c) dz \quad (4.2)$$

Read the two products carefully, because the central hypothesis of this project is visible in them. In Equation 4.1 the distribution over cell t is conditioned on *every cell already emitted*, so the model can in principle know how many boxes it has placed. In Equation 4.2 the cells are conditionally independent given z : once the latent is drawn, no cell knows what any other cell decided.

A valid Sokoban level requires *exactly* one player, four boxes and four goals. That is a constraint on the *joint* distribution over cells. A factorisation that makes cells conditionally independent can match the correct *marginal* rate of boxes without ever producing the correct *number* in any single sample.

A valid level needs exactly 1 player, 4 boxes and 4 goals. Only one of these two can check.

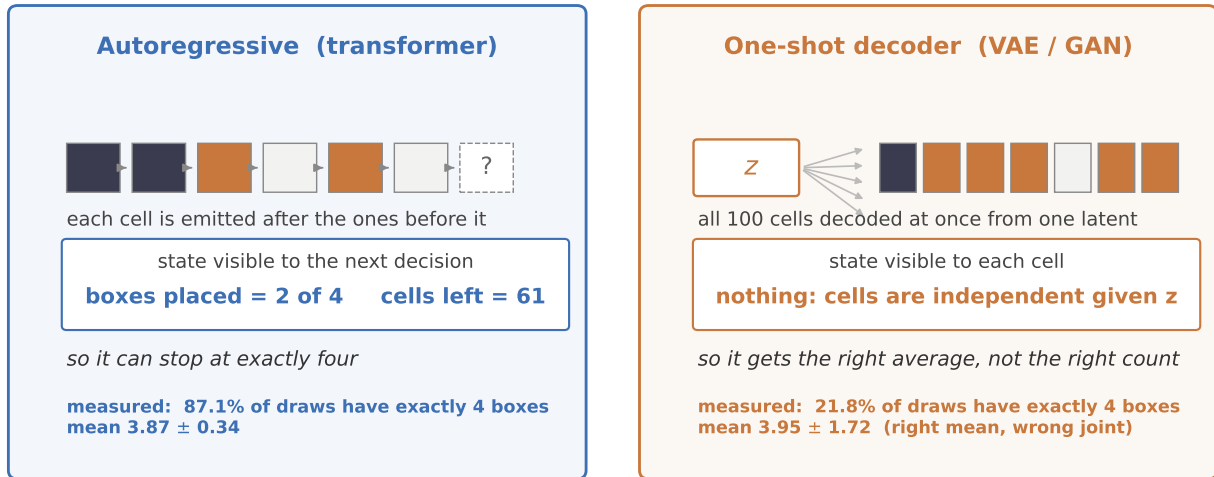


Figure 4.1: The counting hypothesis, and the measured outcome. An autoregressive decoder sees its own partial output, so it can stop at four boxes; a one-shot decoder draws one latent and then decodes all 100 cells independently, so it has nothing to count with. The percentages are measured over raw draws, valid or not.

4.2 Transformer (primary system)

A decoder-only transformer written from scratch: $d_{\text{model}} = 384$, 6 layers, 6 attention heads, $d_{\text{ff}} = 1536$, learned positional embeddings, pre-norm residual blocks, GELU activations, dropout 0.1, and tied input/output embeddings, for a total of 10,734,336 **parameters**.

We deliberately do *not* use a fine-tuned pretrained model as the primary system, so that the three-family comparison is roughly parameter-fair (10.7M against 1.1M and 1.1M) rather than pitting a stock 82M

pretrained model against 1M-parameter decoders. Pretraining is investigated separately as an ablation in Section 4.5.

The loss is next-token cross-entropy **masked over the caption prefix**, so the model is trained to predict the grid given the caption, not to predict the caption itself.

4.3 Conditional variational autoencoder

An MLP encoder and decoder over the $5 \times 10 \times 10$ one-hot tensor, latent dimension 32, hidden width 512, for a total of 1,098,292 **parameters**.

Two design choices are worth stating because the obvious defaults are wrong here:

- **Per-cell categorical cross-entropy, not MSE.** Mean squared error on a one-hot tile tensor treats the categorical channel axis as a metric space, as if “wall” were numerically halfway between “floor” and “box”. It is a common and wrong default.
- **Free bits rather than KL annealing** to prevent posterior collapse, with a floor of 0.05 nats per latent dimension. It is one line of code with no schedule to tune. The measured KL was 8.8 nats, far above the 1.6-nat floor, so the posterior did not collapse.

MLPs rather than convolutions: a 10×10 grid does not factor cleanly for strided convolutions, and at 100 cells convolutions buy very little while costing shape bugs.

4.3.1 Two decoding protocols, both reported

The VAE is reported under **two** decoding protocols, and both are mandatory:

- **Argmax:** take the most likely tile in each cell independently. This is the standard choice and it fails catastrophically, for a reason that is worth understanding: the argmax of a product of independent marginals is *not* the mode of the joint distribution, and rare tiles are precisely what gets annihilated. Boxes occupy 4 of 100 cells; floor or wall wins almost everywhere.
- **Per-cell sampling:** draw each cell from its own categorical distribution. This preserves the marginal tile rates and produces roughly the right *number* of boxes in the wrong *places*.

Reporting argmax alone would be an unfair protocol and would overstate the conclusion, so both appear as separate rows in every table.

4.4 Conditional generative adversarial network

An MLP generator and discriminator, 1,082,357 **parameters** combined, using straight-through Gumbel-softmax discretisation with the temperature annealed from 1.0 to 0.3, plus one-sided label smoothing at 0.9.

Discretisation is not optional here. Training on continuous logits and taking the argmax only at sampling time is *not* a valid option: the discriminator can then separate real one-hot tensors from soft generator output purely by per-cell entropy, training collapses onto that trivial discriminator, and the generator learns nothing at all about layout.

This family was hard time-boxed at three hours. Mode collapse is reported as a finding, not treated as a failure to be fixed.

4.5 DistilGPT-2 ablation: does pretraining transfer?

The ablation asks a specific question: does natural-language pretraining help in a domain that shares *no vocabulary* with the pretraining corpus?

To make the question sharp, the setup discards the 50257×768 token embedding matrix and the tied language-model head, and re-initialises both for our 48-symbol vocabulary. Nothing lexical can transfer. Only the attention and MLP blocks survive, leaving 43,352,064 parameters.

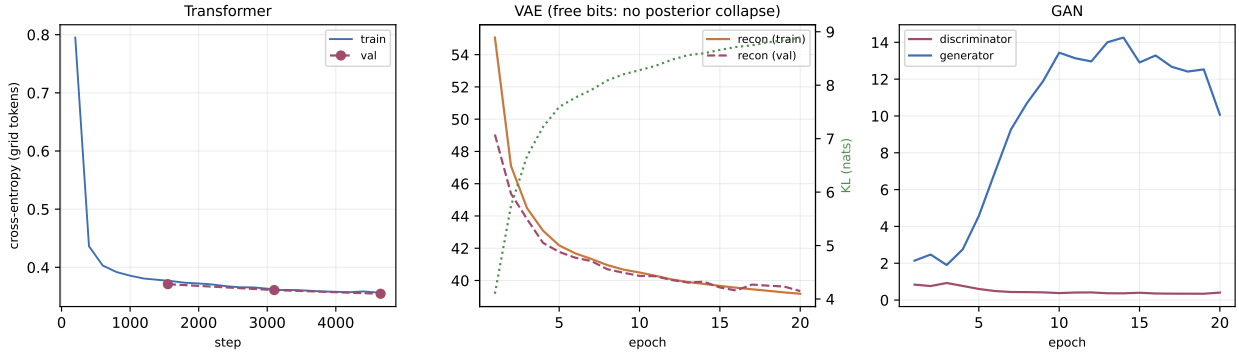


Figure 4.2: Training behaviour of the three families. *Left:* the transformer converges smoothly with train and validation loss tracking each other, so it is not overfitting. *Centre:* the VAE’s reconstruction loss falls while the KL term rises to 8.8 nats, well above the 1.6-nat free-bits floor, confirming the posterior did not collapse. *Right:* the GAN’s generator loss climbs steadily while the discriminator loss stays flat and low, which is the signature of a discriminator that has won. This is mode collapse, and it is reported rather than tuned away.

Table 4.1: The four models side by side. Parameter counts and training times are read from `results/train_*.json`; all runs used bf16 on one RTX 3050 (6 GB).

	Transformer	Cond. VAE	Cond. GAN	DistilGPT-2
Architecture	decoder-only LM, from scratch	MLP en-coder/decoder	MLP generator/discriminator	pretrained body, new vocabulary
Parameters	10,734,336	1,098,292	1,082,357	43,352,064
Conditioning	caption text prefix	caption vector (10-d)	caption vector (10-d)	caption text prefix
Output	110 characters, one at a time	$5 \times 10 \times 10$ logits, one shot	$5 \times 10 \times 10$ logits, one shot	110 characters, one at a time
Optimiser	AdamW, wd 0.01	Adam	Adam, $\beta = (0.5, 0.999)$	AdamW
Learning rate	$3e-4$ (untuned)	$1e-3$ (untuned)	$2e-4$ (untuned)	$3e-4$ (untuned)
Batch / epochs	64 / 3	128 / 20	128 / 20	32 / 3
Train time (seed 1337)	661 s	98 s	153 s	4479 s

Hyperparameters were chosen once and never swept, for every model. We did not try $1e-4$ before settling on $3e-4$. They are labelled *untuned* throughout rather than justified after the fact; an invented justification is the kind of thing that does not survive a viva.

4.6 The counting problem, stated precisely

The remainder of this chapter contains the mechanism that the central claim rests on. Constrained decoding and repair both exist to *separate* two questions that a single accuracy number would fuse together: can the model count, and does the model understand the puzzle?

A structurally valid level needs a complete wall border, exactly one @, exactly four \$ and exactly four .. Nothing about maximum-likelihood training enforces this. The transformer gets close on its own, since 82.6% of unaided draws are valid, but “close” is not a guarantee, and the failures are not random: they concentrate in exactly the region of the distribution the model is least sure about.

4.7 Constrained decoding: a hard rule on the logits

Constrained decoding replaces the sampling loop with a manual one that applies an additive mask to the logits at every step, before the softmax. Three rules are applied:

1. **Border forcing.** Any cell on the outer ring is forced to #, and the eleventh character of each row is forced to a newline.
2. **Quota masking.** Any tile whose quota is exhausted is masked out. Once four boxes have been emitted, \$ becomes unreachable.
3. **Feasibility forcing.** This is the rule that actually matters. Let

$$r = (1 - n_{\text{player}}) + (4 - n_{\text{box}}) + (4 - n_{\text{goal}}) \quad (4.3)$$

be the number of special tiles still owed, and let ℓ be the number of interior cells still to be emitted. When $\ell \leq r$, mask out # and space, so that *only* specials can be emitted for the rest of the grid.

Quota masking alone enforces “*at most* four boxes”, not “*exactly* four”. Without feasibility forcing, a run can reach the last few cells still owing a box and simply never place it. With nine specials in 64 interior cells the rule rarely binds, which is precisely the trap. An implementation without it looks perfectly correct on a handful of samples and fails in the tail.

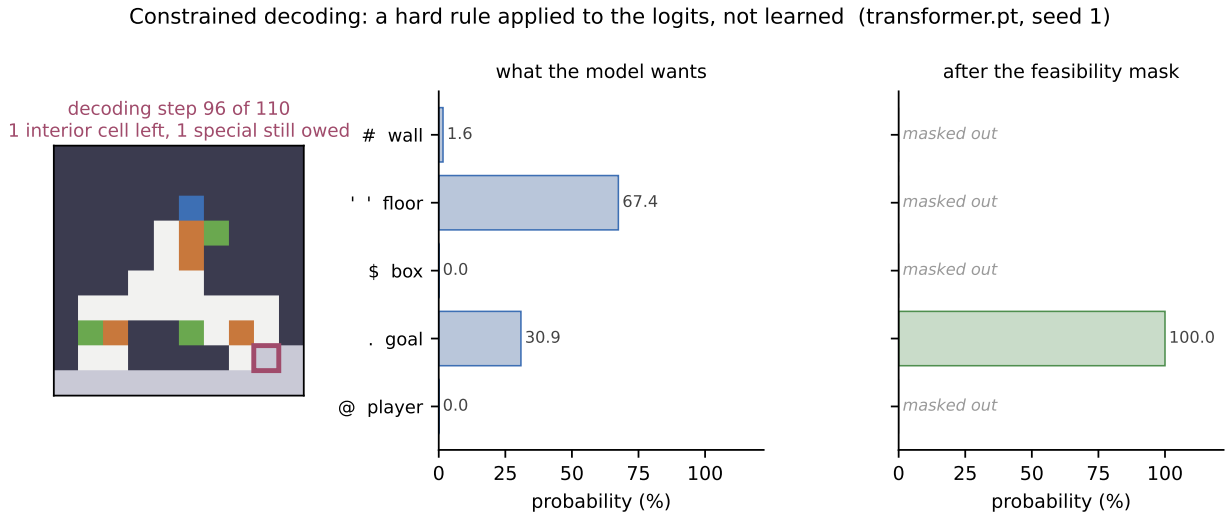


Figure 4.3: Feasibility forcing caught in the act, using the trained checkpoint and real model probabilities. At decoding step 96 the model would put 67.4% of its probability mass on “floor” and only 30.9% on “goal”. But one interior cell remains and one goal is still owed, so the mask removes wall, floor, box and player, and the renormalised distribution puts 100% on “goal”. The pale grey region is the part of the grid not yet emitted; the outlined cell is the one being decided.

4.7.1 How often does it actually fire?

Because the rule is instrumented, the **forced-token rate** is a free measurement of how well the model learned to count on its own. Across the primary transformer’s evaluation sweep, feasibility forcing fired on **25.0% of levels** and supplied a mean of only **0.378 forced tokens per level** out of 110. Quota masking touched 34.4 tokens per level on average, but quota masking is the cheap half of the rule.

The interpretation is that the model had largely learned to count unaided; forcing supplies the last box.

4.7.2 The honest cost

Mask-then-renormalise at each step does *not* sample from the model’s true distribution conditioned on the valid set. It is a greedy local approximation to it, and it therefore shifts the sampling distribution. Section 7.4

shows this cost appearing as a measurable drop in solvability, which is exactly why both the constrained and unconstrained rows are reported.

4.8 Repair: projecting an invalid grid onto a valid one

The one-shot models cannot use a per-step mask, because there are no steps. Instead, any structurally invalid grid is **projected onto the nearest valid grid using the model’s own predicted probabilities**:

1. Force the border to wall.
2. Drop excess boxes, goals and players lowest-probability-first.
3. Fill deficits highest-probability-first.
4. Never place a special tile on the border.

Repair is idempotent and is the identity on already-valid grids.

Repair is what makes the comparison interpretable. Post-repair solvability answers “does this model understand spatial structure?” while structural validity answers “can this model count?” Without repair, a reader cannot tell whether the GAN is bad at Sokoban or merely bad at arithmetic, and as Section 7.4 shows, the answer is not the one the raw numbers suggest.

Evaluation Methodology

5.1 The solver as a decision procedure

Every number in this report ultimately comes from the solver, so the solver is validated before anything else is believed.

Levels are represented as integer bitmasks (walls, goals, boxes) plus a single player cell index. Successors are **pushes**, not single steps. For state hashing the player’s exact cell is irrelevant and only its reachable region matters, so the state key is (minimum reachable cell, box mask). Without this canonicalisation, states differing only by player position within one region are distinct and the search inflates by roughly the region size; measured on the 1,000 real test levels, the starting region averages 19.1 cells.

The heuristic precomputes, per goal, a backward breadth-first map of the minimum number of pushes needed to bring a box to that goal, ignoring other boxes but respecting walls, and then takes the minimum-cost assignment of boxes to goals. It is admissible, because each push moves one box one cell, and strictly tighter than sum-of-Manhattan because it respects walls.

5.1.1 Two sound prunes, and nothing else

- A cell is **dead** if a box placed there can reach no goal under the relaxation above. A box on a dead square is a proven deadlock. This subsumes ordinary corner deadlocks.
- A box is **frozen** if it is blocked along both axes, where a blocker is a wall, a dead square, or another frozen box. A frozen box that is not on a goal is a proven deadlock.

Nothing heuristic is admitted. An unsound prune produces *false proofs of unsolvability*, which is the worst failure mode available here: it corrupts the headline number invisibly, and in the direction that flatters it.

`unsolvable` is returned **only** when the open set empties before any budget is hit. Every cap hit is reported as `timeout`. Collapsing “timed out” into “unsolvable” would convert an absence of evidence into a false proof.

5.2 Solver validation

Table 5.1: Solver validation, run before any model was evaluated (`results/solver_validation.json`). Node cap 200,000, wall-clock cap 10 s.

Check	Result
Solve rate on 1,000 real held-out test levels	100.00%
False “unsolvable” verdicts on real levels	0
Returned solutions that replay to a solved state	1000/1000
Soundness ablation (both prunes disabled, 10× node cap, 200 levels)	0 verdict and 0 length disagreements
Same ablation on three baseline distributions (450 levels)	0 disagreements
Push length vs published solutions (926 comparable levels)	ours $\leq$ theirs on 926/926
Median / p99 nodes expanded	176 / 3,458
Median / p99 wall-clock time	66 ms / 1.4 s

The soundness ablation is the check that matters most. It re-runs the search with both prunes *disabled* and a ten-times larger node budget, and compares verdicts. Zero disagreements means the prunes never discarded a state that mattered. It was extended to 450 levels drawn from the three baseline generators, whose wall densities and failure modes differ sharply from real levels, again with zero disagreements.

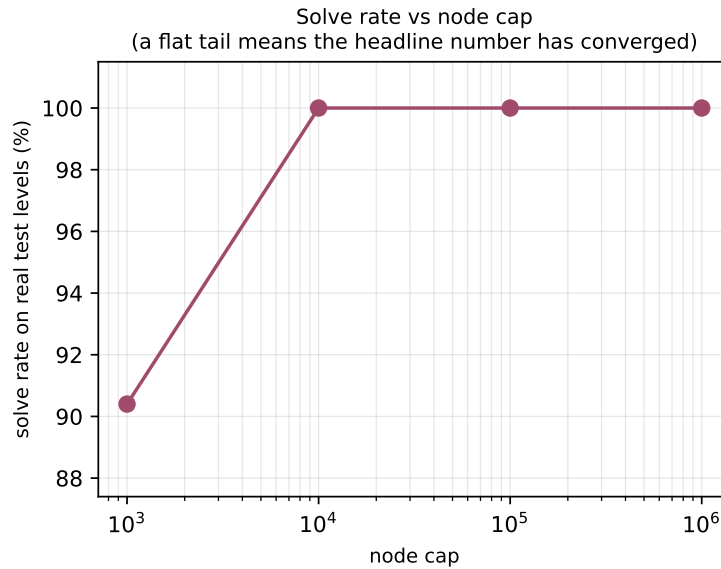


Figure 5.1: The results are converged, not budget-limited. At a node cap of 1,000 the solver still times out on 9.6% of real levels; at 10,000 it reaches 100%, and it stays flat all the way to 1,000,000. Every headline number in this report uses a cap of 200,000, comfortably inside the flat region.

5.2.1 Reconstructed move length is an upper bound

Because push-optimal search discards the player’s exact position, the move length reconstructed by walking the player between consecutive pushes is an *upper bound*, not the optimum. Measured against exact move-optimal search on 196 levels: mean excess +40.5%, exactly optimal on only 6.6% of levels, and *zero* bound violations.

That gap is far too large to ignore, so the achieved solution length used in the controllability metric is measured with the **exact move-optimal solver**, matching the move units used in the captions, rather than with the cheap reconstruction.

5.3 Metrics

Table 5.2: The metrics used throughout Chapter 7.

Metric	Definition
Structural validity %	Fraction of <i>raw draws</i> with a complete wall border and exactly (1,4,4) player, boxes, goals.
Solvable %	Fraction of <i>all samples drawn</i> that the solver proves solvable.
Solvable valid %	Fraction of <i>structurally valid</i> samples proven solvable. Reported separately from validity, never merged.
Timeout %	Fraction on which the solver hit a budget. Never folded into “unsolvable”.
Novelty	Percentage of generated levels that are not an exact copy of any training level, checked over all 8 dihedral transforms.
NN distance	Mean cell-wise Hamming distance (over 100 tiles) to the nearest training level.
Diversity	Mean pairwise Hamming distance within the generated set.
Controllability	Per attribute: the fraction of samples landing in the caption’s requested bin. For solution length, the Spearman rank correlation between requested and achieved.

Novelty and diversity are only interpretable against a reference. On 500 held-out *real* levels the nearest-neighbour distance is 11.92 and the diversity is 38.81; those two numbers anchor every novelty and diversity figure in this report.

5.4 Baselines

Four non-learned baselines constrain how the headline number may be read. They are not there to be beaten; they are there to bound the interpretation.

- **Random placement.** Uniform interior walls at the training-median density, then random special tiles. The floor.
- **Open room.** Border walls only, no interior walls at all. This baseline exists to *invalidate* the headline number if it can: an empty room has no interior walls to freeze a box against, so it should be easy to solve.
- **Rule-based.** Adds a floor-connectivity check and places boxes and goals only on live squares. Hand-written domain knowledge.
- **Retrieval.** Samples a training level verbatim. This makes the complementary point: solvability alone is trivially maximised by copying.

Held-out real Boxoban levels provide the ceiling row.

5.5 Statistical protocol

For each condition we draw until 500 structurally valid levels are collected, or until a hard cap of 100,000 draws is reached, and we report **samples drawn** as its own column. Only valid samples reach the solver, so drawing tens of thousands of one-shot samples costs seconds.

- All proportions carry **Wilson 95% score intervals**, which behave correctly near 0% and 100% where the normal approximation does not.
- Pairwise comparisons use two-proportion z -tests.
- A “solvable given valid” percentage on a denominator below 30 is **flagged inline as noise**. At 2% validity that denominator is ten levels, and a percentage there is noise a reader will misread as signal.
- Three training seeds are run on the primary transformer and on the DistilGPT-2 ablation. **Every other row is a single run and says so.**

Controllability uses a fixed prompt suite stored in the repository: five solution-length targets $\times$ two density settings $\times$ two connectivity settings $\times$ two clustering settings $\times$ 15 samples = 600 prompts. A separate out-of-distribution suite requests 90, 110 and 150 moves.

Implementation

6.1 Pipeline

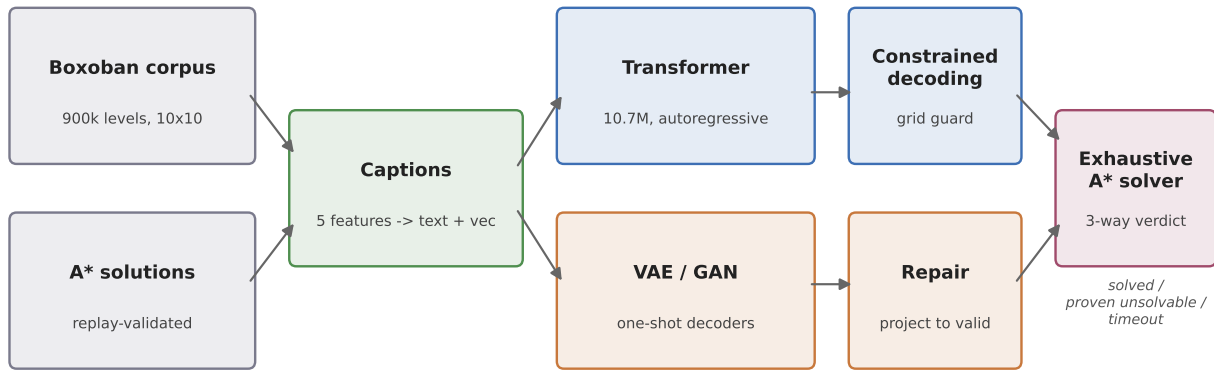


Figure 6.1: The full system. Levels and replay-validated A* solutions feed a caption builder that emits both a text caption and a 10-dimensional vector. The autoregressive branch generates cell by cell and passes through constrained decoding; the one-shot branch decodes in a single step and passes through repair. Everything converges on the exhaustive solver, which returns one of three verdicts.

6.2 Stack and hardware

Python 3.13 with PyTorch (CUDA), `transformers` for the DistilGPT-2 ablation, NumPy, SciPy and Matplotlib. All training and sampling ran on a single **NVIDIA RTX 3050 laptop GPU with 6 GB of VRAM**, in bf16 mixed precision. The solver is pure Python operating on integer bitmasks, parallelised across CPU workers; the full evaluation sweep over 18 conditions took 1,338 seconds.

Model sizes were chosen to fit that budget honestly. The DistilGPT-2 ablation uses batch size 32 rather than 64 purely because 43M parameters do not fit alongside 10.7M-scale activations on a 6 GB card.

6.3 Repository layout

Table 6.1: Module layout.

<code>sokogen/data/</code>	Boxoban parsing, replay-validated solutions, caption construction, vocabulary.
<code>sokogen/solver/</code>	Bitmask grids, sound deadlock detection, exhaustive A*.
<code>sokogen/models/</code>	From-scratch transformer, conditional VAE, conditional GAN, DistilGPT-2 adapter.
<code>sokogen/decoding/</code>	Constrained decoding (the grid guard) and repair.
<code>sokogen/baselines/</code>	Random, open room, rule-based, retrieval.
<code>sokogen/eval/</code>	Metrics, evaluation harness, tables, figures.
<code>scripts/</code>	00–07: the full pipeline in order.
<code>results/</code>	JSON artifacts, generated levels with per-level solver verdicts, tables.

6.4 Reproducibility

The pipeline runs as a fixed sequence of gated scripts:

```
python scripts/00_verify_data.py          # GATE 1:  verify every data assumption
python scripts/01_build_dataset.py         # GATE 3:  captions and datasets
python scripts/02_validate_solver.py      # GATE 2:  solver + soundness ablation
python scripts/03_train.py --model {transformer,vae,gan,distilgpt2}
python scripts/04_generate.py              # GPU sampling
python scripts/05_evaluate.py              # CPU solver sweep → tables + figures
python scripts/06_seed_variance.py         # three seeds per arm
python scripts/07_verify_paper.py          # re-derive every quoted number
```

Seed 1337 is used throughout and every script accepts `--seed`. Every result JSON embeds its configuration, its seed and the git commit SHA that produced it, and every table in this report is regenerated from those artifacts.

`07_verify_paper.py` re-derives every numeric claim in the accompanying manuscript directly from the committed artifacts and fails if any one of them disagrees. It currently reports 108/108 **claims verified**. The same discipline governs this report: no number appears here that is not traceable to a JSON artifact under `results/`.

6.5 Testing

The solver tests were written *before* the solver. They include roughly eleven hand-built fixtures with known verdicts and exact push-optimal lengths, differential testing against an independently written reference solver that shares no code and no representation, and a check that every “solved” verdict comes with an action string that replays to a solved state.

One operational note recorded during the work is worth repeating: the evaluation sweep must not be run while a GPU job is active. The GPU heats the package, CPU cores throttle, and solver throughput silently halves mid-sweep, corrupting the timing measurements without producing any error.

Results and Discussion

Eighteen conditions were sampled and solved. We read the results as six findings.

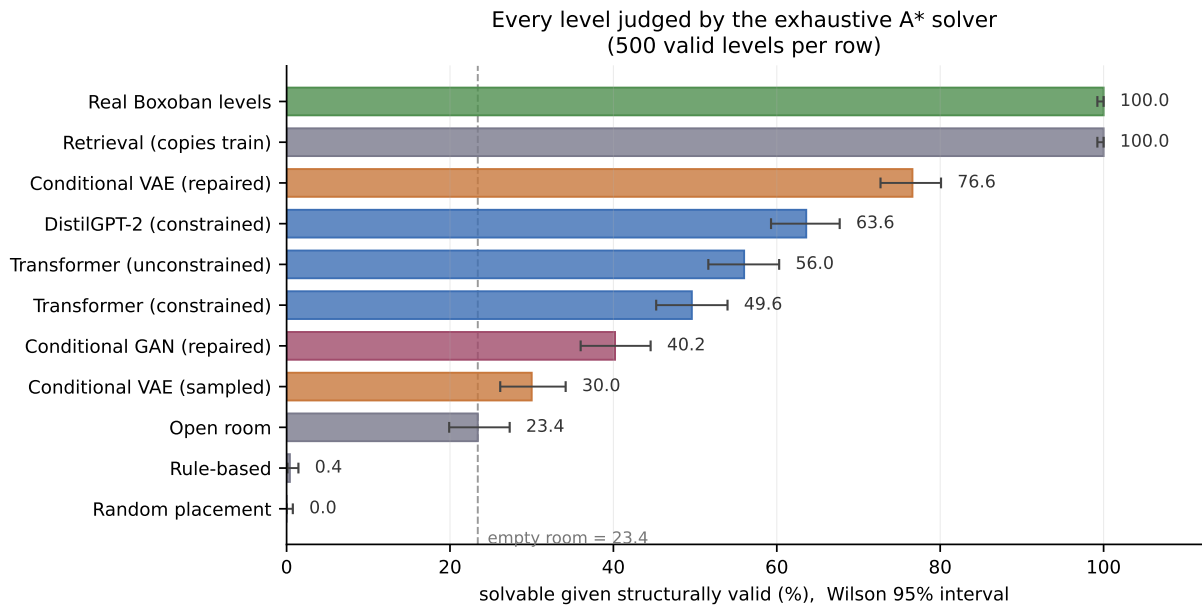


Figure 7.1: Solvability of every condition, judged by the exhaustive solver on 500 structurally valid levels each. The dashed line is the empty-room baseline. Two rows at 100% are not achievements: retrieval copies training levels verbatim, and “real Boxoban levels” is the ceiling.

7.1 Main results

Table 7.1: Main results, part 1: validity and solvability. Percentages carry Wilson 95% intervals. Structural validity and solvability are reported separately and **never merged**. Every row is a single training run; see Section 7.7 for what that licenses.

Model	Struct. valid %	Solvable %	Solvable valid %	Time- out %	Draws for 500 valid
Random placement	100.0	0.0	0.0	0.0	512
Open room	100.0	23.4	23.4 [19.9, 27.3]	0.0	512
Rule-based	100.0	0.4	0.4	0.0	512
Retrieval	100.0	100.0	100.0	0.0	512
Cond. GAN (raw)	8.6 [8.0, 9.4]	4.3	50.2 [45.8, 54.6]	0.0	6,144
Cond. GAN (repaired)	100.0	40.2	40.2 [36.0, 44.6]	0.0	512
Cond. VAE (argmax)	0.0	n/a	n/a	n/a	100,000
Cond. VAE (sampled)	1.7 [1.6, 1.9]	0.5	30.0 [26.1, 34.2]	0.0	29,184
Cond. VAE (repaired)	100.0	76.6	76.6 [72.7, 80.1]	0.0	512
Transformer (unconstrained)	82.6 [80.2, 84.8]	46.3	56.0 [51.6, 60.3]	0.0	1,024
Transformer (constrained)	100.0	49.6	49.6 [45.2, 54.0]	0.0	512
DistilGPT-2 (constrained)	100.0	63.6	63.6 [59.4, 67.8]	0.0	512
Real Boxoban levels	100.0	100.0	100.0	0.0	512

Table 7.2: Main results, part 2: novelty, diversity and cost. Reference distributions on 500 held-out real levels: nearest-neighbour distance 11.92, diversity 38.81. Novelty and diversity are only interpretable against those two numbers.

Model	Novel %	NN dist.	Diversity	Params	Train (s)
Random placement	100.0	23.97	39.20	n/a	n/a
Open room	100.0	20.78	16.22	n/a	n/a
Rule-based	100.0	21.55	38.14	n/a	n/a
Retrieval	0.0	0.00	38.73	n/a	n/a
Cond. GAN (raw)	100.0	11.63	34.18	1,082,357	153
Cond. GAN (repaired)	100.0	12.60	36.79	1,082,357	153
Cond. VAE (argmax)	n/a	n/a	n/a	1,098,292	98
Cond. VAE (sampled)	100.0	12.84	38.54	1,098,292	98
Cond. VAE (repaired)	100.0	11.08	37.82	1,098,292	98
Transformer (unconstrained)	100.0	12.65	39.01	10,734,336	661
Transformer (constrained)	100.0	12.63	38.86	10,734,336	661
DistilGPT-2 (constrained)	100.0	12.18	38.69	43,352,064	4479
Real Boxoban levels	100.0	11.96	38.76	n/a	n/a

7.2 Finding 1: autoregressive models can count; one-shot decoders cannot

The counting gap is by far the largest effect in the table, and it is structural rather than a matter of capacity.

- The transformer reaches 82.6% structural validity **with no constraint at all**, needing 1,024 draws to yield 500 valid levels.
- The conditional VAE under argmax decoding yields **zero** valid levels in 100,000 draws.
- Sampling the VAE per cell instead of taking the argmax raises validity only to 1.7%, requiring 29,184 draws.
- The GAN reaches 8.6%, requiring 6,144 draws.

The tile-count histograms show *why*, far more convincingly than any percentage can.

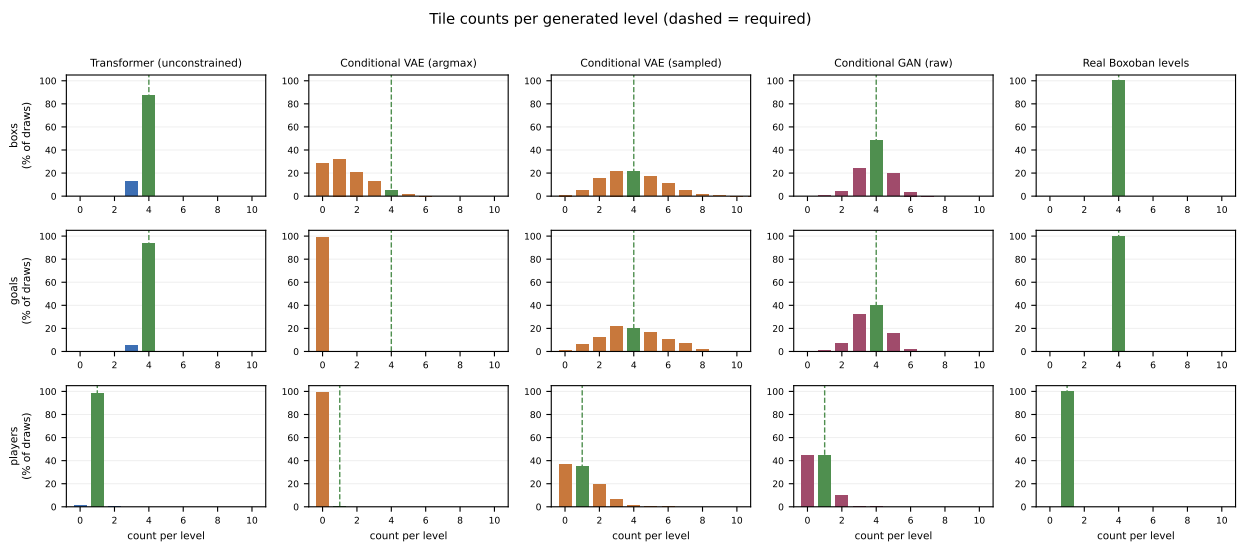


Figure 7.2: Tile counts per generated level, computed over **raw draws, valid or not**; restricting to valid levels would define the counting failure away. The dashed line marks the required count. The transformer’s distribution is a spike at four. The VAE under argmax collapses to a near-empty room. VAE sampling recovers the right marginal rate with the wrong joint.

The numbers behind that figure are the clearest statement of the hypothesis. The transformer: 87.1% of raw draws carry exactly four boxes, mean 3.87 ± 0.34 . The VAE under argmax: mean 1.38 ± 1.23 boxes and 0.01 ± 0.12 goals, because the per-cell argmax of a product of marginals picks floor or wall almost everywhere, because boxes occupy 4/100 cells. VAE sampling restores the marginal rates almost exactly (mean 3.95 ± 1.72 boxes, 4.05 ± 1.85 goals) but only 21.8% of samples carry exactly four: **the right average, the wrong joint.**

Constrained decoding closes the gap deterministically at 100% validity, and with a very light touch: feasibility forcing fires on 25.0% of levels and supplies a mean of 0.378 forced tokens. The model had largely learned to count; forcing supplies the last box.



Figure 7.3: What the failure actually looks like. Each row interpolates between two points in the VAE’s latent space and decodes by argmax. The whole path is a plausible, clean, *empty* room, with one or two boxes at most and no goals at all. The model is not producing noise; it is producing something that looks like a level and contains no puzzle.

7.3 Finding 2: solvability does not follow from validity

Every non-learned baseline is 100% structurally valid by construction and still nearly worthless. Random placement is solvable 0.0% of the time and the rule-based generator 0.4%, despite the latter enforcing floor connectivity and placing boxes only on live squares.

Interior walls are what kill them. At the training-median wall density the floor becomes a maze of narrow corridors, and a box pushed into one freezes against the walls.

The **open-room** baseline constrains how the headline number may be read. With no interior walls at all it is already solvable 23.4% of the time. The transformer’s 49.6% is significantly higher, by +26.2 points ($z = 8.60$, $p < 10^{-16}$), so the headline is not an artifact of an easy task. But a reader given only “49.6% solvable” would substantially overestimate what has been shown.

The **retrieval** baseline makes the complementary point: 100% solvable, 0% novel, nearest-neighbour distance exactly 0.00.

Solvability alone is a degenerate objective, maximised by copying. The results table must be read as a **joint constraint** over solvability, difficulty control and novelty, never as a single ranking.

Can the model count, and does it understand the puzzle?

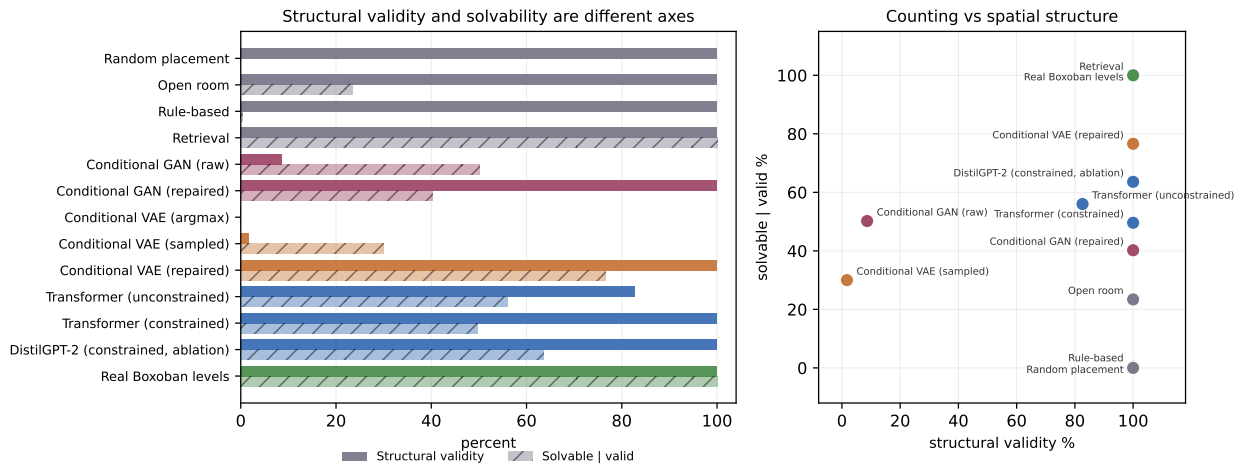


Figure 7.4: *Left:* structural validity (solid) and solvability given validity (hatched) side by side. *Right:* the same data as a scatter. The one-shot families sit at the far left, because they cannot count, while reaching solvability comparable to the transformer once they are valid. A family can be strong on one axis and weak on the other, which is why the two are never merged into a single number.

Table 7.3: Pairwise two-proportion z -tests on solvable | valid.

Comparison	Diff. (pp)	z	p
Transformer (constr.) vs Open room	+26.2	8.60	$< 10^{-16}$
Transformer (constr.) vs Rule-based	+49.2	17.97	$< 10^{-16}$
Transformer (constr.) vs Transformer (unconstr.)	-6.4	-2.03	4.27×10^{-2}
Transformer (unconstr.) vs Cond. VAE (sampled)	+26.0	8.30	$< 10^{-16}$
Transformer (constr.) vs Real Boxoban levels	-50.4	-18.35	$< 10^{-16}$

7.4 Finding 3: repair separates counting from spatial understanding

The most surprising row in the whole table is the repaired VAE. Projecting its invalid output onto the nearest valid grid yields 76.6% solvability, well *above* the transformer’s 49.6%, with diversity (37.8) close to the real-level reference (38.8) and no exact copies. The repaired GAN reaches 40.2%.

The VAE is not bad at Sokoban. It is bad at arithmetic. Its per-cell marginals already encode where boxes and goals plausibly go; repair simply reads them off and places the right *number* of tiles at the most probable positions. What autoregressive generation buys is the ability to satisfy a counting constraint *while sampling*, not a better model of spatial structure. Merging validity and solvability into one number would have hidden this result entirely.

A second, smaller inversion points the same way. The *unconstrained* transformer is more solvable given validity (56.0%) than the constrained one (49.6%; $z = -2.03$, $p = 0.043$). The explanation is selection: unconstrained sampling keeps only the 82.6% of levels the model got right unaided, which are its more typical and more solvable outputs. Constrained decoding additionally rescues the levels it would have botched, and those are worse levels. This is exactly the distribution shift predicted in Section 4.6 appearing as a measurable cost, and it is why both rows are reported. On the overall rate, solvable out of *all* samples drawn, constrained decoding still wins, 49.6% against 46.3%.

7.5 Finding 4: controllability splits along a predictable line

Control is reported per attribute, never as a single number, and the split is sharp.

The constrained transformer hits the caption’s wall-density bin 88.8% of the time, connectivity 80.0% and box clustering 76.3%, against a 50% chance rate confirmed by the unconditioned real-level control (48.8, 53.2, 52.2). But **difficulty**, the binned solution length, is hit only 43.3% of the time against a 33% chance

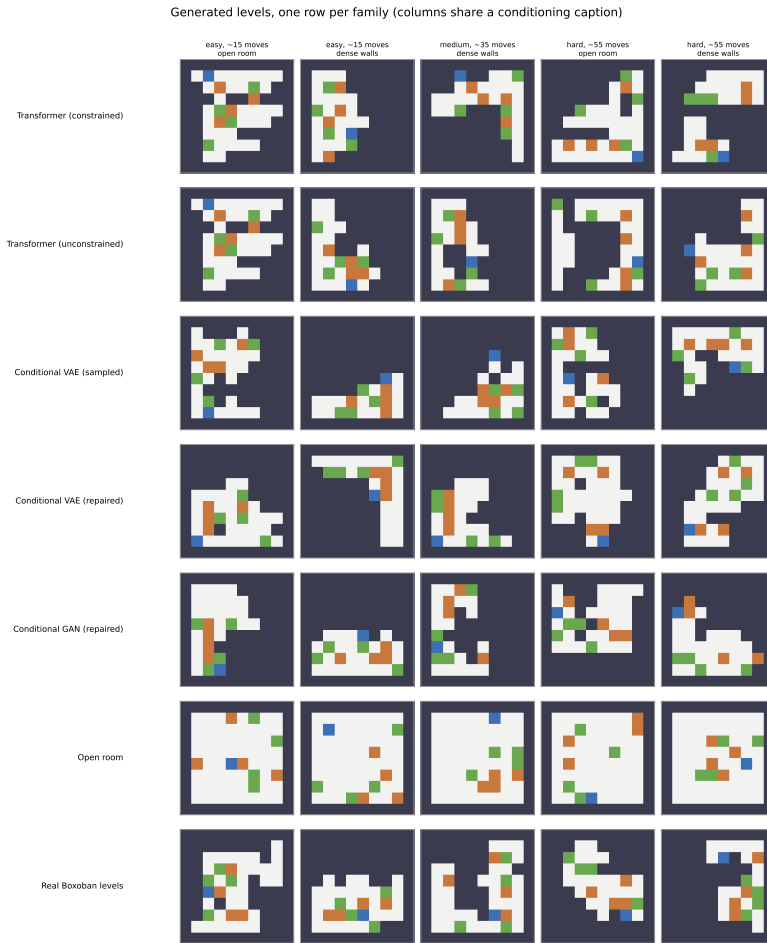


Figure 7.5: Generated levels, one row per family; columns share a conditioning caption. Note the open-room row, which is what a 23.4% solvability baseline actually looks like, and the bottom row of real levels for comparison.

Table 7.4: Controllability per attribute, for the key rows. Bin accuracy against a 50% chance rate for the two-bin attributes and 33% for difficulty. ρ is the Spearman correlation between requested and achieved solution length.

Model	Diffic.	Density	Connect.	Cluster.	Length ρ	Censor. %
Transformer (unconstr.)	41.3	88.3	84.9	78.6	0.516	47.8
Transformer (constr.)	43.3	88.8	80.0	76.3	0.484	55.2
DistilGPT-2 (constr.)	53.5	91.5	85.5	86.2	0.673	37.8
Cond. VAE (repaired)	46.2	88.5	74.2	82.3	0.389	27.0
Cond. GAN (repaired)	39.0	85.7	64.0	53.8	0.074	59.8
Real levels (<i>control</i>)	31.0	48.8	53.2	52.2	-0.048	1.2

rate, with a rank correlation of 0.484 between requested and achieved length.

The dividing line was predicted before the experiment ran. Wall density, connectivity and clustering are **computable directly from the grid**, so a model that reproduces surface statistics gets them nearly for free. **Solution length is not computable without search.** It is the only conditioned attribute that requires the model to have learned something about the *puzzle* rather than about its *pixels*, and it is the one the model controls worst.

That correlation is also **censored**: 55.2% of suite samples have no achieved length at all, because they were proven unsolvable, and the surviving subsample is biased toward easy levels, the direction that inflates ρ . The honest reading is that 0.484 is an *upper bound* on a weak effect.

Table 7.5: Temperature sweep on the constrained transformer.

Temperature	Solvable valid %	Diversity	Novel %
0.6	61.2 [56.9, 65.4]	35.82	100.0
0.8	53.8 [49.4, 58.1]	37.34	100.0
1.0	51.4 [47.0, 55.8]	38.86	100.0
1.2	41.4 [37.2, 45.8]	39.00	100.0
1.5	34.8 [30.8, 39.1]	38.03	100.0

7.6 Finding 5: temperature trades solvability against diversity

The trade-off is clean and monotone: solvability falls from 61.2% at $T = 0.6$ to 34.8% at $T = 1.5$, while diversity rises from 35.8 to a peak of 39.0 at $T = 1.2$. $T = 1.5$ is **Pareto-dominated**: it loses solvability without buying diversity back. At $T = 1.0$ the model already matches the real-level diversity reference (38.9 against 38.8).

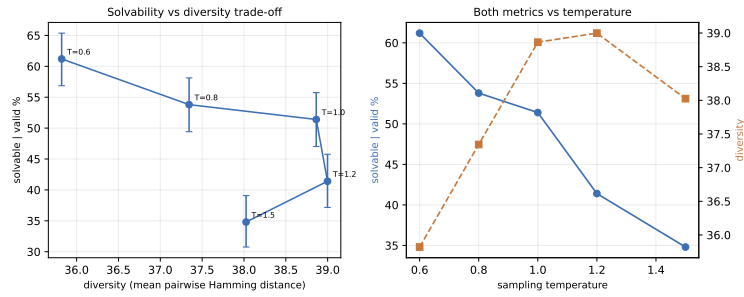


Figure 7.6: Solvability against diversity across sampling temperatures. Bars are Wilson 95% intervals. The point at $T = 1.5$ sits inside the frontier.

7.7 Finding 6: seed variance is large, and it bounds every other claim

We trained the primary transformer configuration **three times**, changing only the random seed, and sampled and solved each run identically.

Table 7.6: Three independent training runs per configuration, seed the only difference. Individual run values in small type.

Model	Struct. valid %	Solvable valid %	Validation loss
Transformer	100.0 $\pm$ 0.0	42.7 $\pm$ 7.0	0.3565 $\pm$ 0.0049
(primary)	100.0, 100.0, 100.0	46.6, 34.6, 47.0	0.3546, 0.3621, 0.3529
DistilGPT-2	100.0 $\pm$ 0.0	64.2 $\pm$ 1.8	0.3414 $\pm$ 0.0006
(ablation)	100.0, 100.0, 100.0	63.6, 62.8, 66.2	0.3415, 0.3407, 0.3418

Structural validity is perfectly stable at 100.0% $\pm$ 0.0, unsurprisingly, since constrained decoding guarantees it, and diversity is stable at 38.60 $\pm$ 0.31. Solvability is **not**: 46.6%, 34.6%, 47.0%.

Validation loss is a poor proxy for the property being evaluated. It moves by ± 0.005 across three runs whose solvability moves by ± 7.0 points. Anyone selecting a checkpoint by validation loss in this setting is selecting close to at random with respect to playability.

This number governs how firmly the rest of the table may be read:

- The comparison against the baselines **survives easily**. Even the worst of the three seeds (34.6%) is far above open room (23.4%) and rule-based (0.4%).

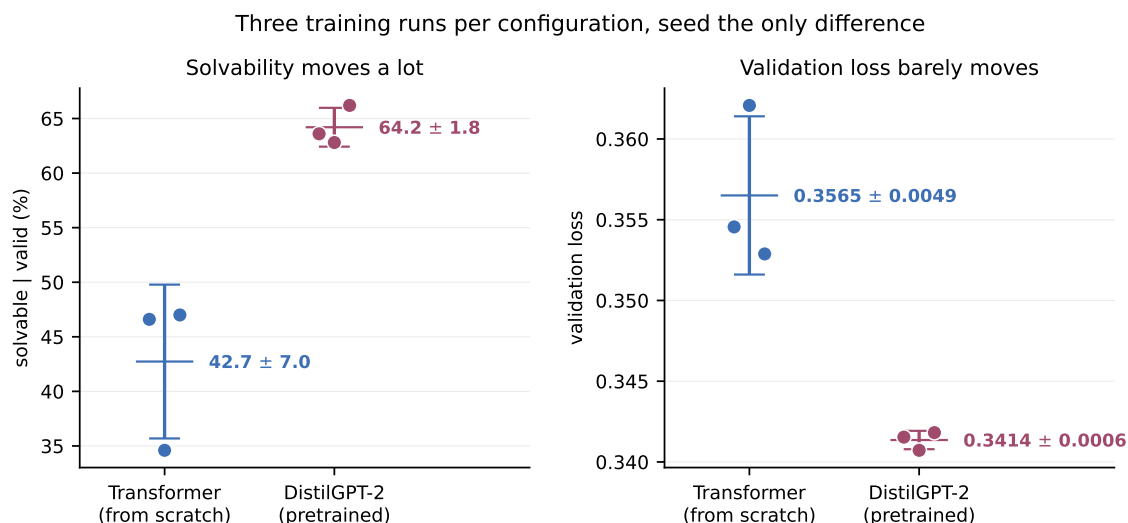


Figure 7.7: The same six training runs viewed two ways. Solvability, the property we actually care about, moves by ± 7.0 points for the from-scratch transformer. Validation loss across the same three runs moves by ± 0.005 . The metric that is cheap to measure is not tracking the metric that matters.

- The constrained-versus-unconstrained gap (6.4 points) is *smaller* than the seed spread. It is a within-checkpoint decoding comparison, so training-seed variance does not apply to it directly, but it is measured on one model and should be treated as indicative only.
- **Every other row in this report is single-seed.** Given a ± 7 point spread on the headline metric, no row-to-row difference under roughly 15 points should be read as established without repeated runs.

7.8 Finding 7: pretraining transfers across a vocabulary boundary

Because the from-scratch model’s own seed spread is ± 7.0 points, a single-run comparison against DistilGPT-2 would prove nothing. So the ablation was trained three times as well.

Across three seeds it reaches **64.2% ± 1.8** solvability against the from-scratch model’s 42.7% ± 7.0 , a 21.5-point gap. **The two sets of runs do not overlap at all** (62.8–66.2 against 34.6–47.0), which is the strongest separation obtainable from three runs per arm: the exact permutation test returns $p = 0.05$, its floor at this sample size, and a Welch t -test gives $t = 5.12$, $p = 0.028$.

It also needs less rescuing from the constrained decoder (10.0% ± 3.4 of levels against 33.7% ± 18.9) and controls solution length better ($\rho = 0.673$ against 0.484).

Pretrained sequence-modelling machinery **does transfer** to a domain that shares no vocabulary with the pretraining corpus. The embedding matrix and LM head were discarded and re-initialised; only attention and MLP blocks survived, and they were enough.

A second, unplanned observation is arguably more interesting than the mean shift: the pretrained model is far more **stable** across seeds, with a standard deviation of 1.8 points against 7.0, a 4.0 $\times$ reduction. We flag this as an *observation*, not a result: with three runs per arm a variance test has almost no power, and Levene’s test is unsurprisingly inconclusive ($p = 0.50$). If it held up at larger n , it would suggest that part of what pretraining buys here is a better-conditioned *initialisation* rather than transferred knowledge.

The cost is unambiguous: 4.0 $\times$ the parameters (43.4M against 10.7M) and 6.8 $\times$ the training time (3897 s against 573 s, averaged over the same three runs).

7.9 Failure analysis: where every family breaks

Negative results are reported here as findings. None of them were tuned away.

7.9.1 Extrapolation fails completely

The out-of-distribution suite requests solution lengths of 90, 110 and 150 moves, against a training distribution whose 99th percentile is 67 moves and whose maximum is 130.

Every family ignores the request.

Table 7.7: Out-of-distribution requests. The models produce roughly 30-move levels when asked for roughly 116. Real Boxoban levels sampled against the same captions are the correct null, since they are not conditioned at all.

Model	OOD ρ	Censor. %	Mean requested	Mean achieved
Transformer (constr.)	−0.394	60.0	115.8	29.5
DistilGPT-2 (constr.)	−0.274	51.7	120.0	30.4
Cond. VAE (repaired)	−0.059	55.0	117.8	42.3
Cond. GAN (repaired)	−0.131	62.5	117.1	24.9
Real levels (<i>null</i>)	0.131	0.8	116.9	30.3

The constrained transformer produces levels averaging 29.5 achieved moves when asked for 115.8, with a rank correlation of −0.394, pointing the wrong way. Models reproduce the training length distribution regardless of what the caption asks for.

Length conditioning learned in-distribution does not extend past the data. This is the expected result and we report it as such, but it is worth stating plainly: a model conditioned on “150 moves” has learned an association between a phrase and a region of its training distribution, not a mechanism for producing long puzzles.

7.9.2 The one-shot failure mode is a plausible, empty room

The VAE’s argmax output is *not* noise. It is a clean, empty, unplayable room with roughly one box and no goals whatsoever (Figure 7.3).

It fails the constraint in the most systematic way available, and it fails *silently*: nothing about the output looks malformed, there is simply no puzzle in it. A soft metric comparing tile histograms in aggregate would not obviously flag this, which is a small but real argument for verification over proxies.

7.9.3 Hand-written domain knowledge is not a shortcut

The rule-based generator, which adds floor-connectivity checks and live-square placement, achieves 0.4% solvability, *worse than an empty room*. Two alternative explanations were checked before accepting that number:

1. **Is the solver wrong?** No. Re-solving 450 baseline levels with deadlock pruning disabled at a 10× node cap gives identical verdicts.
2. **Do the levels start in a deadlock?** No. Additionally rejecting frozen start states moves the rate by less than one point. The levels die *after* the first push, not before it.

Real Boxoban levels reach 100% solvability at the same wall density only because they are built by **reverse play from an already-solved state**, which is a fundamentally different construction from sampling walls and then dropping boxes onto them. Domain knowledge applied in the wrong direction buys nothing.

7.9.4 GAN mode collapse

The GAN’s training curves (Figure 4.2, right) show the generator loss climbing while the discriminator loss stays flat and low: the discriminator has won. Its raw structural validity is 8.6% and its nearest-neighbour distance (11.63) sits below the real-level reference (11.92), consistent with reduced coverage. The family was hard time-boxed at three hours and this is reported as an outcome, not repaired.

Limitations, Ethics and Conclusion

8.1 Limitations

Stated plainly, because each of these bounds a claim made above.

- **Constrained decoding shifts the sampling distribution.** Mask-then-renormalise at each step is a greedy local approximation to sampling from the model’s distribution conditioned on the valid set, not the thing itself.
- **Conditioning channels differ across families.** The transformer is conditioned on caption *text* and the one-shot models on a 10-dimensional *vector*, so the family comparison is not perfectly controlled on conditioning.
- **Reconstructed move length is an upper bound**, quantified at +40.5% mean excess. We avoid it where it matters by using exact move-optimal search for controllability.
- **A single game at a single fixed size.** 10×10 , four boxes, four goals. Nothing here shows the finding generalises to other tile games or other grid sizes.
- **Large seed variance.** Only the transformer and the DistilGPT-2 ablation were run with three seeds. Every other row is a single run, and differences smaller than roughly 15 points between those rows are *not* established.
- **Untuned hyperparameters.** Learning rate, batch size and epoch count were chosen once and never swept, for any model.
- **Inverse-binomial sampling.** Because we draw until 500 valid levels are collected, the structural-validity intervals are close approximations rather than exact fixed- n binomial intervals.
- **Censored controllability.** Achieved solution length exists only for solved levels, so the length correlation is computed on a subsample biased toward easy levels. The censoring rate is reported alongside it every time.

8.2 Ethical considerations

The models generate puzzle layouts for a single game. They consume no personal data, produce no text about people, and inform no decisions about people. The Boxoban corpus is procedurally generated and contains no human-authored or personal content.

The one substantive risk is **misreading the verification claim**. “Solver-verified” is a strong guarantee *only within stated bounds*: a 10×10 grid, four boxes, a 200,000-node cap and a 10-second wall-clock cap. Outside those bounds a timeout proves nothing, which is exactly why timeouts are a separate column rather than folded into “unsolvable”. Presenting a bounded decision procedure as an unbounded guarantee is the failure mode to avoid.

8.3 Conclusion

We evaluated three generative model families using an exhaustive A* solver as a decision procedure rather than a proxy, on a solver validated first: 100.00% solve rate on 1,000 real levels, zero false proofs of unsolvability, and zero verdict disagreements in a soundness ablation across four level distributions. Three claims hold.

1. **Autoregressive generation satisfies counting constraints that one-shot decoders cannot**, at 82.6%

structural validity unaided against *zero* valid levels in 100,000 VAE draws. Constrained decoding closes the gap deterministically, and the forced-token rate shows it only has to supply the last box.

2. **Solvability does not follow from validity**, and the open-room baseline at 23.4% bounds how much of a headline number comes from the task rather than from the model.
3. **The two axes come apart the other way as well**. Once its counting errors are repaired the VAE is *more* solvable (76.6%) than the transformer (49.6%), locating the autoregressive advantage in counting rather than in spatial understanding.

Measuring across seeds proved essential: solvability moves by ± 7.0 points across three runs while validation loss moves by ± 0.005 , and repeating the pretraining ablation under the same protocol settles it at $64.2\% \pm 1.8$ against $42.7\% \pm 7.0$ with no overlap between the two sets of runs.

The most interesting negative result is the controllability split. Attributes computable from the grid are controlled well; solution length, which requires search to evaluate, is controlled weakly ($\rho = 0.484$, censored) and not at all out of distribution. **A model conditioned on “hard” learns what hard levels *look like*, not what makes them hard**, and that gap is invisible to any metric that does not run the game.

8.4 Future work

- **Search-in-the-loop training**. The clearest gap is solution-length control. Using solver feedback as a training signal, whether rejection sampling against a length target or a preference objective over solver verdicts, attacks it directly.
- **More seeds, fewer claims**. The variance-reduction observation under pretraining needs many more than three runs per arm to test properly.
- **Matched conditioning channels**. Feeding the one-shot models caption *text* through a small encoder would remove the one uncontrolled variable in the family comparison.
- **Other grid sizes and other tile games**, to test whether the counting result is a property of autoregressive decoding in general or of this particular grid.
- **Exact constrained sampling**. Replacing greedy mask-then-renormalise with a method that samples correctly from the constrained distribution would remove the measured cost documented in Section 7.4.

References

- [1] G. Todd, S. Earle, M. U. Nasir, M. C. Green and J. Togelius, “Level Generation Through Large Language Models,” in *Proc. 18th Int. Conf. on the Foundations of Digital Games (FDG)*, 2023. *The work this project replicates and extends.*
- [2] S. Sudhakaran, M. González-Duque, C. Glanois, M. Freiburger, E. Najarro and S. Risi, “MarioGPT: Open-Ended Text2Level Generation through Large Language Models,” in *Advances in Neural Information Processing Systems*, 2023.
- [3] A. Summerville et al., “Procedural Content Generation via Machine Learning (PCGML),” *IEEE Trans. Games*, vol. 10, no. 3, 2018.
- [4] A. Guez, M. Mirza, K. Gregor, R. Kabra et al., “An Investigation of Model-Free Planning: Boxoban Levels,” github.com/google-deepmind/boxoban-levels, 2018. *The level corpus.*
- [5] A. Garriga-Alonso, M. Taufeeque and A. Gleave, “Planning behaviour in a recurrent neural network that plays Sokoban,” *ICML 2024 Mechanistic Interpretability Workshop*, 2024. *The A* solution set.*
- [6] S. Racanière, T. Weber, D. Reichert et al., “Imagination-Augmented Agents for Deep Reinforcement Learning,” in *NeurIPS*, 2017. *The level-generation procedure behind Boxoban.*
- [7] J. Culberson, “Sokoban is PSPACE-complete,” Technical Report TR97-02, University of Alberta, 1997.
- [8] A. Junghanns and J. Schaeffer, “Sokoban: Enhancing General Single-Agent Search Methods Using Domain Knowledge,” *Artificial Intelligence*, vol. 129, 2001.
- [9] L. Orseau, L. Lelis, T. Lattimore and T. Weber, “Single-Agent Policy Tree Search with Guarantees,” in *NeurIPS*, 2018.
- [10] M. Chen et al., “Evaluating Large Language Models Trained on Code,” [arXiv:2107.03374](https://arxiv.org/abs/2107.03374), 2021. *HumanEval; execution-based evaluation.*
- [11] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Int. Conf. on Automated Deduction*, 2021.
- [12] T. Yu et al., “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,” in *EMNLP*, 2018.
- [13] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever and M. Welling, “Improving Variational Inference with Inverse Autoregressive Flow,” in *NeurIPS*, 2016. *Free bits.*
- [14] E. Jang, S. Gu and B. Poole, “Categorical Reparameterization with Gumbel-Softmax,” in *ICLR*, 2017.
- [15] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, 1955. *The solver’s admissible heuristic.*
- [16] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *J. Amer. Statist. Assoc.*, vol. 22, 1927. *The confidence intervals used throughout.*

Reproduction and Artifacts

A.1 What is released

All code, configurations, checkpoints, generated levels *with per-level solver verdicts*, and result JSON artifacts accompany this report. Every artifact embeds the configuration, seed and git commit SHA that produced it.

Table A.1: Result artifacts and what each one backs.

<code>data_verification.json</code>	Chapter 3: the alphabet, invariant and solution-unit checks.
<code>dataset_build.json</code>	Caption bins, split sizes, class balance.
<code>solver_validation.json</code>	Table 5.1, the node-cap curve and the move-bound measurement.
<code>train_*.json</code>	Parameter counts, training times and loss histories for all four models and all seeds.
<code>generation.json</code>	Sampling records for all 18 conditions.
<code>evaluation.json</code>	Every number in Chapter 7.
<code>seed_variance*.json</code>	Section 7.7 and Section 7.8.
<code>generated/*.jsonl</code>	Every generated level with its solver verdict.

A.2 Full reproduction

```
pip install -r requirements.txt
git clone https://github.com/google-deepmind/boxoban-levels \
    data_raw/boxoban-levels
# then run scripts 00 through 07 in order (see Chapter 7)
```

Seed 1337 is used throughout; every script accepts `--seed`. Tests run with `python -m pytest tests/`.

A.3 Statement on numeric provenance

Every number in this report (percentages, confidence intervals, parameter counts, timings, correlations and p -values) was read from a committed JSON artifact under `results/`, or, in the case of Figure 4.3, measured live from the committed transformer checkpoint. No value was estimated, rounded from memory, or written as an illustrative placeholder.